Portfolio Allocation as Layered System Optimization:
Mathematical Modeling, Control Decomposition, and Empirical Evaluation

Ashesh Kaji
System Optimization Methods, New York University
ask9184@nyu.edu

April 2026

Abstract

This report presents the portfolio-allocation project explicitly as a System Optimization Methods study. Rather than treating the empirical work as a collection of finance backtests, the paper models allocation as a sequential stochastic control problem with discrete universe and screen decisions, continuous portfolio weights, transaction-cost control effort, and a learned meta-controller. The mathematical starting point is the Markowitz convex mean–variance program, extended through turnover-regularized optimal control, cardinality-constrained screening, online model selection, and policy-gradient reinforcement learning. The final empirical study evaluates this layered formulation on liquid U.S. equity candidate universes of increasing breadth. The central result is that the optimal control stack is not invariant: a top-100 candidate set favors a 21-day momentum screen followed by equal weighting, a top-250 set favors low-volatility screening followed by equal weighting, and a top-500 set favors volatility-adjusted momentum followed by short-horizon rank-and-hold weighting. A policy-gradient router over the same screen/controller arms recovers stable and interpretable policies, but does not dominate a simple trailing-window adaptive rule. The contribution is therefore methodological as much as empirical: the project shows how convex optimization, combinatorial screening, online control, and reinforcement learning can be integrated into a single layered allocation system, and it identifies where each optimization layer adds value or fails to justify its complexity.

1 Motivation: Why This Is a System Optimization Problem

The project began from the classical convex portfolio optimization problem: estimate expected returns $\mu \in \mathbb{R}^n$ and covariance $\Sigma \succeq 0$, then solve

$$\begin{aligned} \max_{w \in \mathbb{R}^n} \quad & \mu^\top w - \lambda w^\top \Sigma w \\ \text{s.t.} \quad & \mathbf{1}^\top w = 1, \qquad w \geq 0. \end{aligned} \tag{1}$$

This model is a natural starting point for a system optimization course because it is a convex quadratic program on the simplex. But it is not a sufficient model of a live allocation system. In practice, a portfolio controller must repeatedly solve a decision problem under uncertainty, update its active set, trade off exploitation against turnover, and decide whether the controller itself should change when the opportunity set changes.

The study therefore treats portfolio allocation as a closed-loop system. At each rebalance time t, the system observes market information $\mathcal{F}_t$, chooses a candidate universe $\mathcal{U}_t$, applies a screen S_t, chooses a weighting controller C_t, executes a portfolio w_t, and observes net reward after transaction costs. The resulting problem combines four optimization types:

1. **Convex optimization:** continuous weight selection under risk and turnover penalties.
2. **Combinatorial optimization:** cardinality-constrained subset selection over candidate securities.
3. **Sequential control:** repeated portfolio adjustment under path-dependent transaction costs.
4. **Online learning / reinforcement learning:** adaptive routing among screen/controller arms.

The purpose of the final experiment is to determine which layer is doing useful work and when extra optimization complexity is justified.

2 From Static Convex Optimization to Dynamic Control

2.1 Single-period convex model

The Markowitz problem in Equation (1) is convex after converting the maximization into minimization:

$$\min_{w \in \Delta_n} \ \lambda w^\top \Sigma w - \mu^\top w, \qquad \Delta_n = \{w : \mathbf{1}^\top w = 1,\ w \geq 0\}. \tag{2}$$

It provides a clean language for risk aversion and diversification. The Lagrangian for the equality-constrained, unconstrained-sign version is

$$\mathcal{L}(w, \eta) = \lambda w^\top \Sigma w - \mu^\top w + \eta(\mathbf{1}^\top w - 1), \tag{3}$$

with first-order condition

$$2\lambda \Sigma w - \mu + \eta \mathbf{1} = 0. \tag{4}$$

This condition shows the central optimization tradeoff: weights tilt toward expected return only insofar as covariance-scaled risk permits. However, the empirical project repeatedly showed that estimating μ and Σ is only one part of the decision. A controller can fail not because the convex program is badly solved, but because the universe or active set is not aligned with the signal being optimized.

2.2 Turnover as control effort

The first operational extension is to add trading friction. Let w_{t^-} be the portfolio before rebalancing and w_t the target portfolio. A proportional transaction-cost model is

$$\mathrm{TC}_t = \kappa \|w_t - w_{t^-}\|_1, \tag{5}$$

where κ is the one-way cost rate. The corresponding turnover-regularized convex controller is

$$\max_{w_t \in \Delta_n} \quad \alpha_t^\top w_t - \lambda w_t^\top \widehat{\Sigma}_t w_t - \gamma \|w_t - w_{t^-}\|_1. \tag{6}$$

Equation (6) connects directly to dynamic trading models with transaction costs: the current position becomes part of the state, and turnover becomes control effort. Even when a rank-based controller rather than a full convex QP is used, the same control-cost logic remains central because high-churn rank-and-hold policies lose more to friction.

2.3 Sequential objective

The live problem is finite-horizon stochastic optimal control:

$$\max_{\pi} \mathbb{E}_{\pi}\left[\sum_{t=0}^{T-1} \delta^t \left(w_t^\top r_{t+1} - \kappa\|w_t - w_{t^-}\|_1 - \rho\,\mathcal{R}(w_t, \mathcal{F}_t)\right)\right], \tag{7}$$

where w_t is the post-trade portfolio, w_{t^-} is the pre-trade portfolio, r_{t+1} is the next-period return vector, κ is the transaction-cost coefficient, ρ is a risk-penalty coefficient, π maps observed state to actions, δ is a discount factor, and $\mathcal{R}$ is a risk or instability penalty. If the transition model were known, the corresponding Bellman recursion could be written by setting $x_t = (\mathcal{F}_t, w_{t^-})$, $a_t = w_t$, and

$$g(x_t, a_t) = \mathbb{E}\Big[w_t^\top r_{t+1} \mid \mathcal{F}_t\Big] - \kappa\|w_t - w_{t^-}\|_1 - \rho\,\mathcal{R}(w_t, \mathcal{F}_t), \tag{8}$$

so that

$$V_t(x_t) = \max_{a_t \in \mathcal{A}(x_t)} \left\{g(x_t, a_t) + \delta\mathbb{E}[V_{t+1}(x_{t+1}) \mid x_t, a_t]\right\}. \tag{9}$$

Here $\mathcal{A}(x_t)$ encodes the admissible portfolio simplex, eligibility, leverage, and turnover constraints, with terminal value normalized as $V_T(x_T) = 0$ for the finite-horizon evaluation used in the backtests. The implemented study approximates this ideal control problem through interpretable policy classes: screens, rank-and-hold rules, equal weighting, and learned routing. The empirical question is not whether the theoretical Bellman equation can be solved exactly; it is whether decomposition into tractable subproblems produces better and more interpretable allocation decisions.

3 Layered Mathematical Formulation

3.1 Bilevel mixed discrete-continuous view

Let $b \in \{100, 250, 500\}$ index candidate-universe breadth, $j \in \mathcal{J}$ index a screen, and $k \in \mathcal{K}$ index a weighting controller. The full model-selection problem can be written as

$$\begin{aligned} \max_{b,j,k} \quad & \frac{\widehat{\mu}_{b,j,k}}{\widehat{\sigma}_{b,j,k}} - \phi\,\widehat{\mathrm{DD}}_{b,j,k} - \psi\,\widehat{\mathrm{TO}}_{b,j,k} \\ \text{s.t.} \quad & H_{b,j}(t) = S_j(\mathcal{U}_b(t), \mathcal{F}_t), \\ & w_t = C_k(H_{b,j}(t), \mathcal{F}_t, w_{t^-}), \\ & |H_{b,j}(t)| \le K_{\max}, \qquad w_t \in \Delta_{|H_{b,j}(t)|}. \end{aligned} \tag{10}$$

Here $\widehat{\mu}_{b,j,k}$, $\widehat{\sigma}_{b,j,k}$, $\widehat{\mathrm{DD}}_{b,j,k}$, and $\widehat{\mathrm{TO}}_{b,j,k}$ are walk-forward estimates of mean return, volatility, drawdown, and turnover for the stack (b, j, k). This is not a single convex program. It is a mixed discrete-continuous model-selection problem with lower-level portfolio maps embedded in C_k. The empirical design makes it tractable by enumerating a disciplined grid of (b, j, k) arms, evaluating each arm out of sample, and then studying the performance surface.

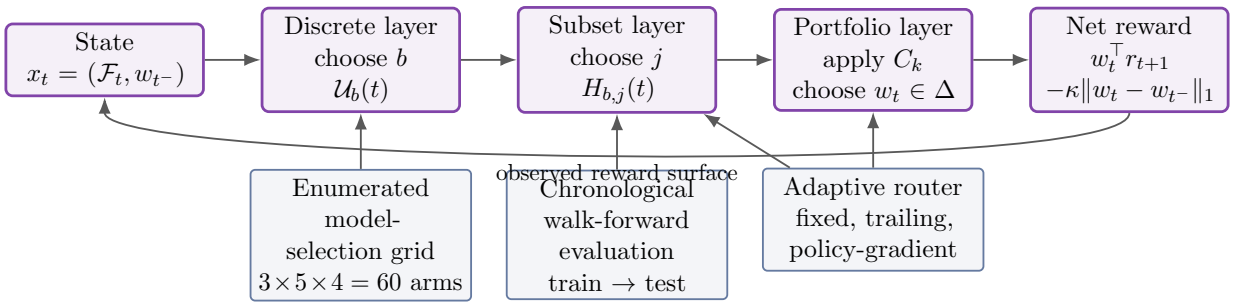

Figure 1: Optimization decomposition used in the study. A difficult mixed discrete-continuous sequential problem is decomposed into enumerated universe/screen/controller arms, walk-forward evaluation, and an adaptive routing layer.

3.2 Screens as constrained projections

Each screen is a set-valued map from the candidate universe into a smaller active set:

$$S_j : (\mathcal{U}_b(t), \mathcal{F}_t) \mapsto H_{b,j}(t), \qquad |H_{b,j}(t)| \le K_{\max}. \tag{11}$$

For a score $q_i(t)$, the top-K screen solves the cardinality problem

$$\max_{z\in\{0,1\}^{|\mathcal{U}|}} \sum_{i\in\mathcal{U}} q_i(t) z_i \quad \text{s.t.} \quad \sum_i z_i \le K. \tag{12}$$

The five score functions tested are

$$q_i^{\text{mom},\tau}(t) = P_i(t)/P_i(t-\tau) - 1, \tag{13}$$

$$q_i^{\text{vam},\tau}(t) = \frac{P_i(t)/P_i(t-\tau) - 1}{\widehat{\sigma}_{i,\tau}(t) + \epsilon}, \tag{14}$$

$$q_i^{\text{lowvol},\tau}(t) = -\widehat{\sigma}_{i,\tau}(t), \tag{15}$$

$$q_i^{\text{cluster}}(t) = q_i^{\text{mom},63}(t) \quad \text{with per-cluster selection caps.} \tag{16}$$

This is where a major system-optimization insight enters the project: screening is not data cleaning. It is a discrete optimization layer that changes the feasible set available to every downstream controller.

3.3 Weighting controllers as restricted policy classes

After screening, the controller chooses weights on the active set H_t. Equal weighting is the minimum-estimation controller:

$$w_{i,t} = \frac{1}{|H_t|}\mathbf{1}\{i \in H_t\}. \tag{17}$$

Rank-and-hold is another cardinality-constrained policy, now inside the screened set:

$$w_{i,t} = \frac{1}{k}\mathbf{1}\left\{i \in \text{TopK}_{\ell\in H_t}\left(P_\ell(t)/P_\ell(t-\tau) - 1\right)\right\}. \tag{18}$$

Although Equation (18) is simpler than a full quadratic program, it is still an optimization rule: it solves a top-k linear ranking objective over the screened active set. The final study compares whether this extra active layer is worth its turnover cost after screening has already concentrated the universe.

4 Learning and Meta-Control Formulation

4.1 Model selection as an online expert problem

Let each screen/controller pair be an arm $a \in \mathcal{A}$ with realized reward $R_t(a)$ in walk-forward period t. A non-learning model-selection baseline chooses

$$a^{\text{train}} = \arg\max_{a \in \mathcal{A}} \sum_{t \in \mathcal{T}_{\text{train}}} R_t(a) \tag{19}$$

and holds that arm in the test period. The trailing-window router chooses

$$a_t^{\text{trail}} = \arg\max_{a \in \mathcal{A}} \sum_{s=t-L}^{t-1} R_s(a), \tag{20}$$

which is an online adaptive rule. This comparator is important because any learned method must beat simple, transparent online optimization before it can be considered practically useful.

4.2 Policy-gradient router

The reinforcement-learning router treats the arm-selection problem as a finite-action Markov decision process. The state s_t contains time features, scope identity, trailing rewards, and the most recent reward vector. The policy is a softmax over arm logits:

$$\pi_\theta(a \mid s_t) = \frac{\exp(f_\theta(s_t)_a)}{\sum_{a' \in \mathcal{A}} \exp(f_\theta(s_t)_{a'})}. \tag{21}$$

The reward is next-period net reward minus a switching penalty:

$$r_t^{\text{router}} = R_t(a_t) - \xi \mathbf{1}\{a_t \neq a_{t-1}\}. \tag{22}$$

The policy-gradient update estimates

$$\nabla_\theta J(\theta) = \mathbb{E}_\pi \left[\sum_t (G_t - b_t) \nabla_\theta \log \pi_\theta(a_t \mid s_t) \right], \tag{23}$$

where G_t is a return-to-go and b_t is a variance-reduction baseline. The important methodological choice is that reinforcement learning is not asked to learn raw portfolio weights from scratch. It is placed at the system level, where the action space consists of interpretable optimization stacks.

5 Experimental Protocol as Optimization Evidence

The final empirical matrix uses three universe scopes, five screens, and four weighting controllers, producing $3 \times 5 \times 4 = 60$ primary arms at the 10 basis point cost slice. Each arm is evaluated through chronology-respecting walk-forward splits: screen membership is computed using only information available through the training end date, and performance is measured on the subsequent evaluation interval. This is the empirical analogue of out-of-sample control evaluation.

Table 1: Layer map of the optimization decomposition and its empirical instantiations.

Layer	Mathematical role	Implementation in final study
Universe breadth	Discrete state/action defining candidate feasible set	Top-100, top-250, and top-500 liquid-equity candidate scopes
Screen	Cardinality-constrained subset optimization	Momentum, volatility-adjusted momentum, low-volatility, cluster-capped momentum top-10 screens
Weighting	Continuous or restricted portfolio policy on simplex	Equal weight and 21/63/126-day rank-and-hold top-5 controllers
Trading friction	Control effort / path dependence	Net reward subtracts proportional turnover cost at 10 bps primary slice
Routing	Online model selection / MDP policy	Train-selected fixed arm, trailing-window router, random router, policy-gradient router, oracle bounds
Validation	Out-of-sample control evaluation	Chronological walk-forward splits, all-trials ledger, model-selection ledger, multiple-comparison diagnostic

6 Optimization Results

6.1 Main solution stacks

The strongest empirical evidence that the layered model matters is that the selected solution stack changes with universe breadth.

Table 2: Best screen/controller stack by candidate-universe breadth. Values are mean walk-forward metrics at 10 bps transaction cost.

Universe	Discrete screen solution	Weighting solution	Splits	Ann. ret.	Ann. vol.	Sharpe	Worst DD
Top-100	21-day momentum top 10	Equal weight	22	20.4%	20.2%	1.474	-44.9%
Top-250	63-day low-volatility top 10	Equal weight	11	11.2%	14.1%	0.859	-10.5%
Top-500	Vol.-adjusted 63-day momentum top 10	21-day rank-and-hold	9	35.1%	20.8%	1.756	-15.5%

The optimization interpretation is direct. If the universe is narrow, screening already performs most of the selection and equal weighting is an adequate solution on the resulting simplex. In the broadest universe, the feasible set contains enough cross-sectional dispersion that a second ranking optimization inside the screened set improves risk-adjusted performance. In the intermediate universe, low-volatility screening dominates, suggesting that the best solution is to reduce noise rather than chase momentum.

6.2 Screen layer: objective functions matter

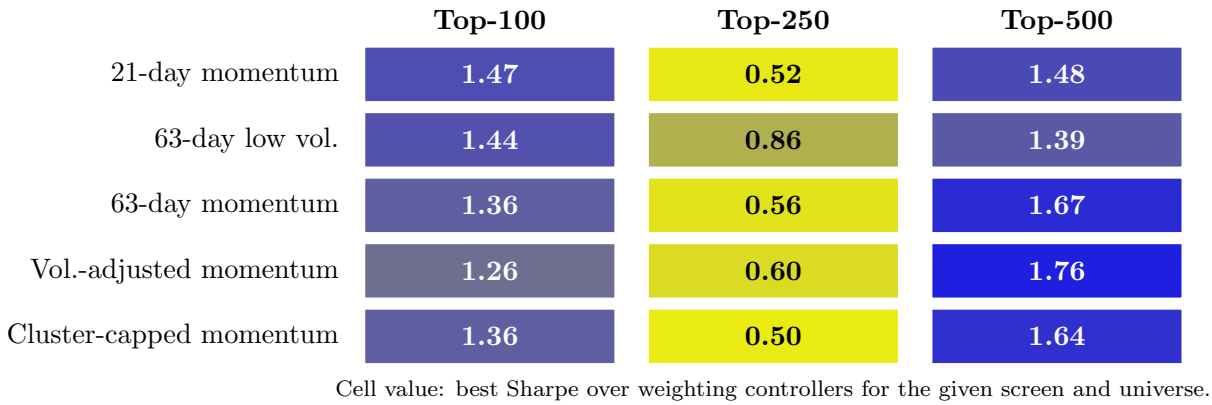

Figure 2: Screen objective functions define different feasible active sets. The heatmap shows that the best screening objective changes by universe breadth.

Figure 2 is the most compact visual evidence that the screen is an optimization layer. The top-250 universe is weak across most momentum screens, but improves under the low-volatility objective. The top-500 universe rewards volatility-adjusted and medium-horizon momentum objectives. Thus, the screen objective changes the feasible set in a way that can dominate the weighting rule.

6.3 Weighting layer: when extra control effort pays

Averaging over screens, equal weighting has the best mean Sharpe in the top-100 and top-250 scopes, while 21-day rank-and-hold has the best mean Sharpe in the top-500 scope. This is a system-level cost-benefit result. Extra optimization effort is not universally valuable; it is valuable when the upstream feasible set is broad enough to contain exploitable dispersion.

Table 3: Controller ablation: best weighting rule averaged over screens.

Universe	Best controller by average screen Sharpe	Mean Sharpe	Best Sharpe	Interpretation
Top-100	Equal weight	1.380	1.474	screen does most selection
Top-250	Equal weight	0.572	0.859	risk filtering beats active ranking
Top-500	21-day rank-and-hold	1.433	1.756	breadth justifies second control layer

The turnover diagnostics support the same conclusion. Equal weight has essentially zero rebalance turnover after screen membership is fixed, while rank-and-hold rules introduce substantial turnover. When active ranking does not add enough signal, the turnover penalty makes the simpler controller superior.

6.4 Model-selection risk and multiple comparisons

Because each universe evaluates 20 screen/controller arms, a winning Sharpe must be interpreted as a selected maximum over many trials:

$$\widehat{a} = \arg\max_{a \in \mathcal{A}} \widehat{\mathrm{SR}}(a). \tag{24}$$

The study therefore reports the winner gap against the second-best arm rather than presenting winners as formal statistical discoveries.

Table 4: Exploratory multiple-comparison diagnostic. Gaps are descriptive, not p-values, because walk-forward folds overlap and arms are not independent.

Universe	Arms tested	Best Sharpe	Second Sharpe	Winner gap
Top-100	20	1.474	1.440	0.034
Top-250	20	0.859	0.755	0.104
Top-500	20	1.756	1.666	0.090

The top-100 winner is fragile because the gap is small. The top-250 and top-500 winners are more informative mechanically, but still should be treated as exploratory. This is why the paper emphasizes mechanisms–screen and controller ablations–rather than only selected winners.

7 Reinforcement Learning as System-Level Optimization

The learned router is evaluated as a meta-controller over already-interpretable optimization arms. This makes the RL component appropriate to the project: it is not a black-box replacement for portfolio theory, but a learned model-selection policy in a structured control system.

Table 5: Integrated policy-gradient router results. Rewards are cumulative test-period rewards at the 10 bps cost slice; RL values are mean ± standard deviation over seven seeds for the best configuration.

Scope	Test periods	RL	Train-fixed	Trailing	Random	Oracle	RL–fixed	RL–trailing
Combined	14	1.213 ± 0.063	1.134	1.543	1.276	2.586	0.079	-0.330
Top-100	8	0.823 ± 0.011	0.827	0.475	0.657	1.355	-0.004	0.348
Top-250	5	0.280 ± 0.068	0.305	0.306	0.296	0.764	-0.026	-0.026
Top-500	4	0.371 ± 0.000	0.371	0.566	0.525	0.932	0.000	-0.196

The RL results are academically useful precisely because they are not overclaimed. The policy-gradient router improves over the train-selected fixed arm in the combined panel and learns stable selections, but it does not dominate the trailing-window router. In optimization terms, the learned policy class and state representation are sufficient to recover some structure in the reward surface, but not sufficient to beat a low-complexity online optimizer that directly exploits recent realized rewards.

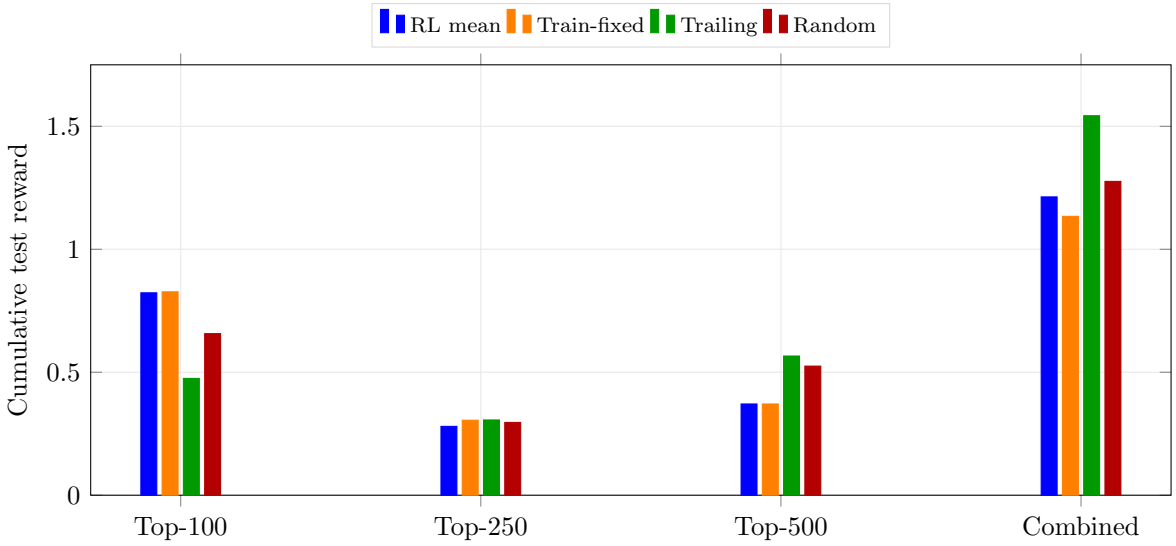

Figure 3: Adaptive model-selection comparison. The trailing-window rule is a strong online optimization baseline; the learned router is informative but not yet dominant.

8 How the Optimization Methods Shaped the Final Study

The final experiment is not a finance study retrofitted into a systems class. It is the result of repeatedly translating portfolio ideas into optimization objects:

1. The project began with Equation (1), a convex quadratic program, but empirical instability motivated simpler restricted policies and turnover-regularized objectives.
2. Transaction costs converted static allocation into a path-dependent control problem, forcing the study to compare net rather than gross performance.
3. Cardinality and universe construction converted the problem into a mixed discrete-continuous system, where feasible-set design can matter more than continuous weight optimization.
4. Walk-forward evaluation turned backtesting into out-of-sample control validation rather than full-sample curve fitting.
5. Reinforcement learning was integrated at the meta-control level, where the action space is structured and interpretable, rather than as an unconstrained black-box allocator.

This progression is the central academic contribution. The mathematics was not ornamental; it determined what had to be measured. In particular, the layered formulation predicted that changing the feasible set could change the optimal controller, and the empirical study found exactly that.

9 Limitations and Next Mathematical Step

The main limitations are also optimization limitations. First, the candidate universes are not fully point-in-time, so the feasible sets $\mathcal{U}_b(t)$ are only approximations to investable historical universes. Second, the screen/controller grid is finite; it approximates the bilevel problem but does not solve the full mixed-integer program in Equation (10). Third, the RL router has few independent test periods per scope, so policy-gradient results should be treated as exploratory. Fourth, market impact is modeled as proportional turnover rather than a nonlinear function of order size and liquidity.

The next rigorous step is not another broad sweep. It is to build point-in-time universe membership and rerun the layered system with more independent walk-forward episodes. Mathematically, this

would improve the fidelity of the feasible set $\mathcal{U}_b(t)$ and provide enough samples to evaluate whether learned routing approximates the online model-selection optimum better than simple trailing-window selection.

10 Conclusion

This report reframes the final portfolio project as a layered system optimization study. The empirical conclusion is that the best allocation method is not just a weighting formula. It is a stack: universe construction, screen objective, weighting controller, and optional router. In the top-100 universe, the solution is simple: momentum screening plus equal weighting. In the top-250 universe, risk filtering is the useful optimization. In the top-500 universe, breadth creates enough dispersion for volatility-adjusted momentum plus short-horizon rank-and-hold to perform best. The learned router confirms that the hierarchy is learnable, but also demonstrates that a transparent trailing-window optimizer remains a difficult baseline.

For System Optimization Methods, the core lesson is that model formulation determines insight. Once portfolio allocation is modeled as a mixed discrete-continuous sequential control problem, the project stops being a search for a single best trading rule and becomes a study of where optimization should be applied: to the feasible set, to the weights, to the control cost, or to the routing policy. That is the practical and academic value of the final study.

A Reproducibility Artifacts

The primary run folder is `findings/2026-04-23-context-aware-grand-study/`. Key artifacts include:

- `metrics/screened_core_summary.csv`, `metrics/screened_universe_winners.csv`, and `metrics/screen_ablation_summary.csv`;
- `metrics/controller_ablation_summary.csv`, `metrics/multiple_testing_summary.csv`, and `metrics/integrated_rl_study_*.csv`;
- `report/SYSTEM_OPTIMIZATION_METHODS_STUDY.tex` and `report/SYSTEM_OPTIMIZATION_METHODS_STUDY.pdf`.

B Controller and Screen Grid

The core grid uses five screens: 21-day momentum top 10, 63-day momentum top 10, 63-day volatility-adjusted momentum top 10, 63-day low-volatility top 10, and 63-day cluster-capped momentum top 10. It uses four weighting controllers: equal weight, 21-day rank-and-hold top 5, 63-day rank-and-hold top 5, and 126-day rank-and-hold top 5. The primary reported cost slice is 10 basis points; 0 and 25 basis point slices are retained for cost-sensitivity diagnostics.

References

[1] Bailey, D. H., and López de Prado, M. (2014). The Deflated Sharpe Ratio: Correcting for Selection Bias, Backtest Overfitting, and Non-Normality. *Journal of Portfolio Management*, 40(5), 94–107.

[2] Cover, T. M. (1991). Universal Portfolios. *Mathematical Finance*, 1(1), 1–29.

[3] DeMiguel, V., Garlappi, L., and Uppal, R. (2009). Optimal Versus Naive Diversification: How Inefficient is the 1/N Portfolio Strategy? *Review of Financial Studies*, 22(5), 1915–1953.

[4] Gârleanu, N., and Pedersen, L. H. (2013). Dynamic Trading with Predictable Returns and Transaction Costs. *Journal of Finance*, 68(6), 2309–2340.

[5] Markowitz, H. (1952). Portfolio Selection. *Journal of Finance*, 7(1), 77–91.

[6] Williams, R. J. (1992). Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. *Machine Learning*, 8(3), 229–256.

[7] Zhang, Z., Zohren, S., and Roberts, S. (2020). Deep Reinforcement Learning for Trading. *Journal of Financial Data Science*, 2(2), 25–40.